

graph, the number of occurrences of a and b up to that edge and store $1/(n_{e,a} + 1)$ and $1/(n_{e,b} + 1)$ in the token corresponding to the edge, where $n_{e,a}$ and $n_{e,b}$ are the corresponding counts. Then given the edge (u, v) , we can use the Match mechanism to find $1/(n_{(u,v),v} + 1)$. Then we will use a feed-forward layer with gated relu activation, constant depth, and constant width to approximate $1/(n_{(u,v),v} + 2)$ and then use the Match mechanism again to find the next edge containing v .

Through the above two functions, the Transformer can simulate the DFS algorithm and hence solve the IsTree problem perfectly. $\square$

4.2.3 Transformers are Strictly More Expressive Than RNNs

The above theorems show the existence of tasks where Transformers require exponentially less memory than RNNs. However, they have not rule out the possibility that there exists a corresponding task where the Transformer will be more redundant and require exponentially more parameters than RNNs. However, the following theorem confirms that such a task doesn't exist for constant-size RNN.

The theorem is in the same vein as the recent work on the CoT for Transformer (Li et al., 2024), which shows the constant size and constant precision Transformer with a polynomial-size position embedding can simulate any polynomial size circuit. The major difference of our theorem is that (1) we consider a Transformer with fixed word and position embedding, hence allowing the parameter number to be logarithmic in the input size, and (2) we consider simulating RNNs, which is a special kind of circuit family and hence we can use more succinct representation utilizing the structural property attached to the recursive process.

Theorem 4.8. *Given input length n , let R is an RNN with word embedding $W^{(E)} \in \mathbb{R}_p^{(n+n_S) \times d}$, where $p = \Theta(\log n)$ is the precision, the constant n_S is the number of special symbols in the vocabulary, the constant d is the embedding dimension. If each recurrent iteration can be computed by a circuit of size $C(n) \leq 2^{p/2}$, and if the RNN produces the final answer after running at most n^A steps of CoT for some constant $A > 0$, then there exist Transformers with $O(\log n)$ -bit precision, $O(C(n))$ parameters and word embedding $[W^{(E)} \quad \mathbf{0}^{(n+n_S) \times d}]$ that can produce the same final answer after running $(C(n) + 1)n^A$ steps of CoT.*

Proof Sketch. The key idea is to encode the RNN circuit into the Transformer's weight and simulate the circuit gate by gate, utilizing the Match mechanism to fetch the input of each gate. In this manner, although the naive circuit to simulate the RNN for n^A steps would require $O(n^A)$ parameter, the Transformer only needs to store one instance of the RNN circuit in its weight and hence we only need $O(P(n) \log \max\{P(n), n\})$ parameter, which is at most polynomial in the parameter size of the RNN. $\square$

5 Enhancing the In-context Retrieval Capability Closes the Representation Gap

In Section 4.2, we show that RNNs are deficient at In-context Retrieval, hence leading to a significant representation gap with Transformers. In this section, we aim to understand: if we enhance the In-context Retrieval capability of RNNs, do RNNs remain to have any representation gap with Transformers? We answer this question by examining two representative approaches to enhance the In-context Retrieval capability, one explicit and one implicit, and show that both ways can close the representation gap between RNNs and Transformers in solving algorithmic problems.

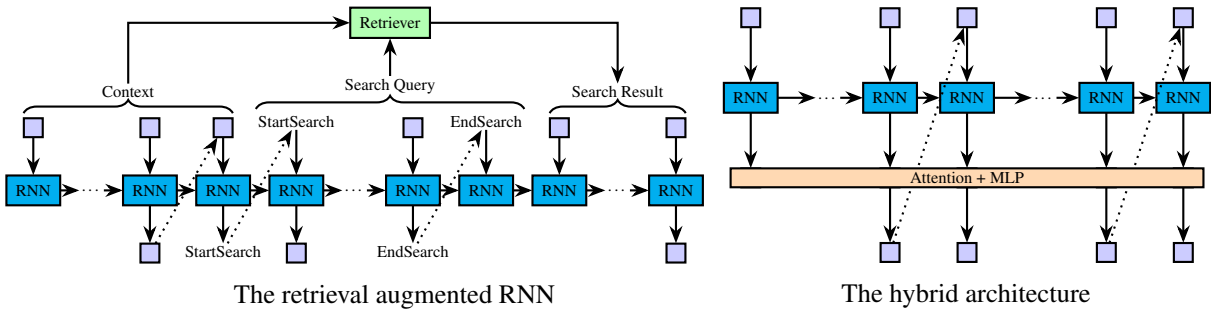


Figure 4: **In-context RAG.** The retrieval augmented RNN (left) and the hybrid architecture (right) close the representation gap between RNNs and Transformers.

5.1 Explicit Retrieval Through Regular Expression

First, we explore the power of RNNs with *Retrieval Augmented Generation* (RAG), which gives an LM the capability to retrieve relevant information to assist generation. In our context, we are specifically interested in allowing LMs to call functions to retrieve information from their context, which we call *In-context Retrieval Augmented Generation* (In-context RAG).

We will first show that adding function calls to associative recall is not enough to close the representation gap between RNNs and Transformers.

Proposition 5.1. *For any RNN family with $O(\log n)$ bit memory and $O(\log n)$ parameter with an oracle to receive results for the AR problem (Definition 4.3) for any queries, for large enough n , the RNN can't solve the index problem (Definition 4.2) with length n in any CoT steps.*

Proof. Consider a special type of index problem where every token at the even position of the input sequence is a special token κ and the rest of the tokens are uniformly random. Then the oracle for the AR problem can be simulated by the RNN by simply outputting the κ when the query is not κ and outputting the third token when the query is κ . However, following similar proof of Theorem 4.6, we can show that the RNN can't solve this special form of index problem with length n in any CoT steps. $\square$

In this light, we need to consider a more general form of In-context Retrieval capability. We specifically consider a special form of In-context RAG that enables an LM to perform regular expression matching because the regular expression is a flexible primitive that can be used to describe a wide range of retrieval tasks and can be implemented efficiently on modern hardware.

Given an LM M with vocabulary V (containing two additional special tokens, $\langle \text{StartSearch} \rangle$ and $\langle \text{EndSearch} \rangle$) and the tokenized input sequence $\mathcal{S}_{\text{in}} \in |V|^{l_0}$, the LM M with In-context RAG generates following sequence of tokenized sequence:

$$\begin{aligned} \mathcal{S}_0 &= \mathcal{S}_{\text{in}}, & s_i^{\text{next}} &= \arg \max_{j \in V} M(\mathcal{S}_i)[j], \\ \mathcal{S}_{i+1} &= \begin{cases} \mathcal{S}_i \oplus s_i^{\text{next}}, & \text{if } s_i^{\text{next}} \neq \langle \text{EndSearch} \rangle \\ \mathcal{S}_i \oplus s_i^{\text{next}} \oplus \text{RETRIEVE}(\mathcal{S}_i), & \text{otherwise.} \end{cases} \end{aligned}$$

Here RETRIEVE looks for the last occurrence of $\langle \text{StartSearch} \rangle$ at position l_s and $\langle \text{EndSearch} \rangle$ in $\mathcal{S}$ at position l_e and treat $\text{Detokenize}(\mathcal{S}_{l_s:l_e})$ as a regular expression, where Detokenize maps the tokenized sequence back to the string, inserting a space between every pair of adjacent tokens. The algorithm then runs a regular expression matching on $\text{Detokenize}(\mathcal{S}_{1:l_s-1})$, finds the first matching substring, and returns the first capturing group according to the regular expression (i.e., content embraced by a pair bracket in the regular expression). While there are many grammar standards of regular expressions, we adhere to the standard specified in the `re` library of Python. That is, we evaluate the following Python code to get the result of the regular expression matching:

```
re.search(pattern, string).group(1)
```

where $\text{Detokenize}(\mathcal{S}_{l_s:l_e})$ is the pattern and $\text{Detokenize}(\mathcal{S}_{1:l_s-1})$ is the string.

The following theorem shows that In-context RAG with regular expressions is powerful enough for RNNs to solve the In-context Retrieval problems in Section 4.2.1 with CoT.

Theorem 5.2. *For task $T \in \{\text{Index}, \text{AR}, c\text{-gram retrieval}, \text{Counting}, \text{IsTree}\}$, there exist constant-size Linear RNNs that can solve T with In-context RAG. For T other than IsTree, $O(1)$ steps of CoT is required and for IsTree, $O(n \log n)$ steps of CoT is required.*

Proof Sketch. For the Index problem, let the RNN output the regular expression $^{\wedge} (? : \backslash S \backslash S^*) \{a\} (\backslash S)$, where $a = k - 1$. For AR, let the RNN output $\backslash b q \backslash b (\backslash S^+) \backslash b$, where q is the number in query. For c -gram retrieval, let the RNN output $\backslash b q_1 \dots q_{c-1} \backslash b (\backslash S^+) \backslash b$, where q_i is the i -th number in the query. For Counting, let the RNN output $(\backslash b v \backslash b) \{k + 1\}$, where v is the query token and k is the query threshold. $\square$

Beyond these simple In-context Retrieval problems, in Theorem 4.7, we have shown that RNNs cannot solve IsTree due to its implicit requirement of In-context Retrieval capability. We now show that In-context RAG can help linear RNN solve IsTree in $O(n)$ CoT steps. See Appendix C.9 for the proof.

Theorem 5.3. *There exists a Linear RNN family with $O(\log n)$ bit memory and $O(\log n)$ parameter, that can solve IsTree of size n with In-context RAG in $O(n)$ CoT steps.*

To further show the power of the explicit retrieval, the following theorem further proves a general result, showing that In-context RAG empowers RNNs with $O(\log n)$ bit memory to simulate polynomial-time Turing machines.

Theorem 5.4. *Given $A, B > 0$, for all polynomial-time Turing machines $T \in \text{TIME}(n^A)$ with B states and vocabulary size B , there exist Linear RNNs with B special symbols, $O(A \log n)$ -bit precision and memory, and $O(AB^2)$ parameters that can output the result of T by running $O(n^A)$ steps of CoT with In-context RAG.*

Proof Sketch. The retrieval augmented RNN can simulate the Turing machine by maintaining the state of the Turing machine and the position of the tape head in its memory and writing down the tape in the context in the form of $c v c$ to indicate the value of the tape at position c is updated to v . Given the input tape $\text{TAPE}[1, i]$, the retrieval augmented RNN will first write down the initial tape in the previous form of $i \text{ TAPE}[1, i]$ using the regular expression used in the Index problem. The RNN can then generate the search queries with forms of $p \ (.) \ p \ . * ? \$$ with p being the pointer to retrieve the information from the context.

By introducing In-context RAG, RNNs are able to use regular expressions to retrieve information from a distance and put relevant information together. Then RNNs can use perform more complex operations locally and output tokens to be retrieved later. $\square$

As a final note, our focus here is to understand the representation power of RNNs given an appropriate RAG, but not to propose a method that immediately leads to practical applications. While the above results show that In-context RAG can close the representation gap between RNNs and Transformers in solving algorithmic problems, a limitation here is that In-context RAG is not an immediate practical solution, as there is no existing training data for this In-context RAG.

5.2 Implicit Retrieval by Appending Just One Transformer Layer

Since Bahdanau et al. (2016), attention mechanisms have been understood as a form of compensation for the fixed memory size of RNNs, allowing the model to attend to the entire context. We show in this section formally that this form of implicit retrieval can close the representation gap between RNNs and Transformers in solving algorithmic problems. We consider the following hybrid architecture, which combines the RNN and the Transformer by appending a single Transformer layer to the RNN output.

Definition 5.5 (Hybrid RNN). A hybrid RNN is a model that consists of an RNN with transition and output function $\mathbf{t}, \mathbf{o}$ and one Transformer layer f , the output of the RNN is used as the input of the Transformer layer and the output of the Transformer layer is used to produce the next token. Concretely, given the input sequence S_{in} , the output of the hybrid architecture is:

$$\mathcal{H}(S) = \text{softmax} \left(W^{(E)} f \left([\mathbf{o}(s_k)]_{k \in [l]} \right) \right)_{:,l},$$

$$\forall k \in [l], s_k = \mathbf{t}(s_{k-1}, \text{Emb}(S)_{:,k}),$$

First, we show that hybrid RNNs can solve the In-context Retrieval problems in Section 4.2.1 without CoT.

Theorem 5.6. *For task $T \in \{\text{Index}, \text{AR}, c\text{-gram retrieval}, \text{Counting}, \text{IsTree}\}$, there exist constant-size hybrid Linear RNNs that can solve T . For T other than IsTree , no CoT is required and for IsTree , $O(n \log n)$ steps of CoT is required.*

Proof Sketch. The proof is similar to the proof of Theorem 4.6, using the appended Transformer layer to simulate the Match function and COUNT function in the RNN. $\square$

Similar to the situation in Section 5.1, the implicit retrieval method can empower the hybrid linear RNN to solve IsTree with CoT. See Appendix C.12 for the proof.

Theorem 5.7. *There exists a hybrid Linear RNN with $O(\log n)$ bit memory and $O(\log n)$ parameter, that can solve IsTree of size n with a chain of thought of length $O(n \log n)$.*

Further, we show that this hybrid architecture with only one attention block is powerful enough to even simulate any polynomial-time Turing machine with CoT.

Theorem 5.8. *Given any constant A, B , for any polynomial-time Turing machine $T \in \text{TIME}(n^A)$ with B states and vocabulary size B , there exists a hybrid Linear RNN (see Definition 5.5) with vocabulary of B special symbol, $O(A \log n)$ bit precision and memory, and $O(AB^2 \log n)$ bit parameters, that can simulate the result of T on any input with length n in $O(n^A)$ CoT steps.*

Proof Sketch. The proof is similar to the proof of Theorem 5.4. Instead of using regular expressions to retrieve the information from the context, the hybrid architecture can use the attention mechanism in the Transformer layer to implement the Match function to retrieve the information from the context. $\square$